

- Introduction
- VHDL Code Structure
- Data types, Data Objects and Operators
- Statements in VHDL
- Standard Architectures
- FSM Design

10.1 Introduction

VHDL is a hardware description language. It is used to describe the behavior of an electronic system, which further enables designer to implement the physical system. VHDL stands for VHSIC Hardware Description Language, where VHSIC is an acronym for Very High Speed Integrated Circuits.

The main purpose of VHDL is to model and synthesize digital circuits. Simulation and testing of the design for the optimum operation can be done using VHDL model of the system. Also, digital integrated circuits for particular operations can be created using VHDL or other hardware description languages. Finally, VHDL code can be used to create actual functional system. Hence, VHDL code can be used either to implement the circuit in a programmable device or can be forwarded for fabrication.

VHDL Invariants

- It is not case sensitive.
- It is not sensitive to white space.
- Comments begin with two consecutive dashes ("--").
- Parenthesis usage is optional in many cases.
- Every statement in VHDL is terminated with a semicolon.
- Statements are inherently concurrent. Only statements placed inside a PROCESS, FUNCTION, or PROCEDURES are executed sequentially.

10.2 VHDL Code Structure

Fundamental VHDL Units

VHDL code comprises of at least the three fundamental sections: LIBRARY Declaration, ENTITY and ARCHITECTURE. LIBRARY is a collection of pre-defined set of codes that can be re-used or shared by various designs. ENTITY specifies the I/O connections of the system. ARCHITECTURE contains the code that describes how the circuit should function.

LIBRARY DECLARATION

The general form is:

```
LIBRARY LIBRARY_NAME;
USE LIBRARY_NAME.PACKAGE_NAME.PACKAGE_PARTS;
```

Example:

```
LIBRARY IEEE;
```

```
USE IEEE.STD_LOGIC_1164.ALL;
```

The libraries *STD* and *WORK* are made visible by default, so they are not required to declare. However, *STD_LOGIC_1164* package of *IEEE* library must be declared when *STD_LOGIC* data type is used in the design. Similarly, for *SIGNED* and *UNSIGNED* data types and its related arithmetic and comparison operations, package *STD_LOGIC_ARITH* of *LIBRARY IEEE* must be declared.

ENTITY

The VHDL ENTITY declaration describes the interface or the external representation of the circuit. An ENTITY is a list of all input and output pins with its specification such as data type and data direction mode.

Its syntax is:

```
ENTITY ENTITY_NAME IS
  PORT (
    PORT_NAME: SIGNAL_MODE SIGNAL_TYPE;
    PORT_NAME: SIGNAL_MODE SIGNAL_TYPE;
    ...
  );
END ENTITY_NAME;
```

Here, *ENTITY_NAME* and *PORT_NAME* are identifiers. The *SIGNAL_MODE* which defines the direction of signal can be *IN*, *OUT*, *INOUT*, or *BUFFER*. *IN* and *OUT* are unidirectional pins, while *INOUT* is bidirectional. The data type or *SIGNAL_TYPE* can be *BIT*, *STD_LOGIC*, *INTEGER*, etc.

Example 1: Entity of AND gate with two inputs each of one bit.

```
ENTITY AND_GATE IS
  PORT (
    IN_A : IN STD_LOGIC;
    IN_B : IN STD_LOGIC;
    OUT_Z : OUT STD_LOGIC
  );
END AND_GATE;
```

Example 2: Entity of 4x1 MUX with each input of three bits

```
ENTITY MUX IS
  PORT (
    A, B, C, D : IN STD_LOGIC_VECTOR(2 DOWNTO 0);
    SEL : IN STD_LOGIC_VECTOR(1 DOWNTO 0);
```

```

        Z : OUT STD_LOGIC_VECTOR(2 DOWNTO 0)
    );
END MUX;

```

ARCHITECTURE

The ARCHITECTURE describes how the circuit should function. It describes the internal implementation of the associated entity. There are several models that are followed by architecture to describe the operation of the circuit.

The general form of ARCHITECTURE is:

```

ARCHITECTURE architecture_name OF entity_name IS
    [declarations]
BEGIN
    [code]
END architecture_name;

```

Here, declarative part is optional and includes signal and constant declarations. Code part includes different VHDL statements describing the system to be designed.

Example 1: ARCHITECTURE of AND gate with two inputs each of one bit.

```

ARCHITECTURE AND_ARCH OF AND_GATE IS
BEGIN
    OUT_Z <= IN_A AND IN_B;
END AND_ARCH;

```

10.3 Data types, Data Objects and Operators

A. DATA TYPES

Pre-Defined Data Types

VHDL contains a series of pre-defined data types. Such data type definitions can be found in various packages or libraries.

- o Package STANDARD of library STD includes BIT, BOOLEAN, INTEGER, and REAL.
- o Package STD_LOGIC_1164 of library IEEE includes STD_LOGIC and STD_ULOGIC.

Various pre-defined data types are listed in the table below:

SN	TYPE	LEVEL/RANGE	DESCRIPTION
1.	BIT	2 LOGIC LEVEL	0, 1
2.	BIT_VECTOR	2 LOGIC LEVEL	0, 1

3.	STD_LOGIC	8 VALUED LOGIC	X, 0, 1, Z, W, L, H, -
4.	STD_LOGIC_VECTOR	8 VALUED LOGIC	X, 0, 1, Z, W, L, H, -
5.	STD_ULOGIC	9 VALUED LOGIC	U, X, 0, 1, Z, W, L, H, -
6.	STD_ULOGIC_VECTOR	9 VALUED LOGIC	U, X, 0, 1, Z, W, L, H, -
7.	BOOLEAN	2 VALUES	TRUE, FALSE
8.	INTEGER	-2147483647 TO +2147483647	32 BIT NUMBER
9.	NATURAL	0 TO +2147483647	
10.	REAL	-1.0E38 TO +1.0E38	
11.	SIGNED	POSITIVE AND NEGATIVE	USED IN ARITHMETIC OPERATIONS
12.	UNSIGNED	POSITIVE	
13.	PHYSICAL	TIME, VOLTAGE	USED IN SIMULATION

Here, various logic levels represent: 'U' – Unresolved, 'X' – Forcing Unknown, '0' – Forcing Low, '1' – Forcing High, 'Z' – High Impedance, 'W' – Weak unknown, 'L' – Weak Low, 'H' – Weak High, '-' – Don't care. STD_LOGIC levels are intended for simulation only. When a node has two STD_LOGIC signals connected, then conflicting logic levels are resolved automatically in case of STD_LOGIC whereas such conflict is not resolved in STD_ULOGIC. For arithmetic operations using STD_LOGIC, packages STD_LOGIC_SIGNED and STD_LOGIC_UNSIGNED must be used.

User Defined Data Types

VHDL allows users to define their own data types. There are two categories of user-defined data types.

o User-Defined Integer Type

General form:

TYPE TYPE_NAME IS RANGE LOW_VALUE TO HIGH_VALUE;

Example:

TYPE TEMPERATURE IS RANGE -125 TO 125;

TYPE MARKS IS RANGE 0 TO 100;

o User-Defined Enumerated Type

General Form:

TYPE TYPE_NAME IS (VALUE1, VALUE2... VALUEN);

Example:

TYPE COLOR IS (RED, GREEN, BLUE, WHITE);

Based on bits requirement encoding of enumerated type is done sequentially and automatically, unless specified.

B. DATA OBJECTS

An object is an item in VHDL that has both name and a specific type. Commonly used data objects are signals, variables and constants.

CONSTANTS are used to assign default values in the code. It can be declared in **PACKAGE**, **ENTITY** or **ARCHITECTURE**. Declaring **CONSTANTS** in **PACKAGE** makes it global, since **PACKAGE** can be used by several entities. If it is declared in an **ENTITY**, it can be shared by all **ARCHITECTURE** that follows that **ENTITY**. When defined within **ARCHITECTURE** the scope of **CONSTANTS** are limited to that **ARCHITECTURE** only.

Declaration:

CONSTANT name:TYPE:= value;

Examples:

CONSTANT high:STD_LOGIC:='1';

CONSTANT count:INTEGER:=10;

SIGNAL is used to pass value in and out of the circuit and within internal units. It simply represents interconnection of circuit. All ports of **ENTITY** are signals by default. The change in the **SIGNAL** may not be updated immediately, since the value is more likely to get updated after the completion of its corresponding **PROCESS**, **FUNCTION** or **PROCEDURE**. Similar to **CONSTANT**, it can be declared in **PACKAGE**, **ENTITY** or **ARCHITECTURE**.

Declaration:

SIGNAL name: TYPE [range] [:= initial_value];

The part inside the square bracket may or may not be present depending upon data types used and requirement of initialization.

Examples:

SIGNAL start: STD_LOGIC:= '0';

SIGNAL count: INTEGER RANGE 0 TO 100;

VARIABLE represents the local information. Its value cannot be passed out directly. The change in value is immediately updated; new value can be promptly used in next line of code. It can be declared and used inside a **PROCESS**, **FUNCTION** or **PROCEDURE**.

Declaration:

VARIABLE name: type [range] [:= initial value];

Examples:

VARIABLE count: **INTEGER**:=0;

VARIABLE a : **STD_LOGIC_VECTOR**(7 **DOWNTO** 0);

C. OPERATORS

The various operators supported by VHDL are tabulated below:

ASSIGNMENT OPERATORS

SN	Operator	Assign Value To	Examples
1.	<=	Signal	X <= '1'; Y <= "101";
2.	:=	Variable, constant, generic, and for initialization	Z := "1001"; Z is a variable
3.	=>	Individual Elements or with OTHERS	W <= (0=>'1', OTHERS =>'0') LSB assigned 1 and others as 0

LOGICAL OPERATORS

SN	Operators	Description/Example	Supported Data Type
1.	NOT	Inverts the signal, High Precedence	BIT STD_LOGIC STD_ULOGIC BIT_VECTOR STD_LOGIC_VECTOR STD_ULOGIC_VECTOR
2.	AND	Result high when both inputs is high	
3.	OR	Result high when one of the inputs is high	
4.	NAND	X <= a NAND b	
5.	NOR	Z <= NOT a NOR B	
6.	XOR	Complements the bit when XORed with 1	
7.	XNOR	Complements the bit when XNORed with 0	

RELATIONAL OPERATORS

SN	Operators	Description
1.	=	Equal to

2.	/=	Not equal to
3.	<	Less than
4.	>	Greater than
5.	<=	Less than or equal to
6.	>=	Greater than or equal to

ARITHMETIC OPERATORS

SN	Operator	Meaning	Description
1.	+	Addition	
2.	-	Subtraction	
3.	*	Multiplication	
4.	/	Division	Limited to powers of two
5.	**	Exponentiation	Limited to powers of two
6.	MOD	Modulus	X MOD Y results value with sign of Y
7.	REM	Remainder	X REM Y results value with sign of X
8.	ABS	Absolute Value	
Avoid Using MOD operator when dealing with negative numbers			

SHIFT OPERATORS

SN	Operator	Meaning	Description
1.	SLL	Shift Left Logic	Zeros are fed from one end and bits are lost from other end. Sign bit never changes in arithmetic shift. "10101" SLL 3 results in "01000"
2.	SRL	Shift Right Logic	
3.	SLA	Shift Left Arithmetic	
4.	SRA	Shift Right Arithmetic	
5.	ROL	Rotate Left	"1001" ROL 2 results in "0110"
6.	ROR	Rotate Right	

CONCATENATION OPERATOR

The concatenation operator (&) is used to combine values of similar data type. The following example will illustrate the use of concatenation operator.

Example:

```

signal A, B : std_logic_vector (3 downto 0);    -- Signal A and B of 4 bits
signal C : std_logic_vector (5 downto 0);       -- Signal C of 6 bits

```



```
signal D : std_logic_vector (7 downto 0);
```

-- Signal D of 8 bits

```
C <= A & "00";
```

-- 4 bits of A and two bits "00" assigned to C

```
D <= B & A;
```

-- 4 bit of A and B combined and assigned to D

10.4 STATEMENTS IN VHDL

A. CONCURRENT STATEMENTS

Concurrent Signal Assignment

Syntax:

```
Target <= expression;
```

Examples:

```
A <= B NAND C;
```

```
X <= (D OR E) AND (F AND G);
```

Conditional Signal Assignment

Syntax:

```
Target <=      expression when condition else
               expression when condition else
               expression;
```

Example:

```
Z <= '1' when (L='0' AND M='0') else
      '1' when (L='1' AND M='1') else
      '0';
```

Selective Signal Assignment

Syntax:

```
with choose_expression select
target <= expression when choices,
         expression when choices;
```

Example:

```
with SEL select
M_OUT <=  A3 when "11",
           A2 when "10",
           A1 when "01",
```

Example:

```

LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

ENTITY HALF_ADDER IS
    PORT(
        A, B: IN STD_LOGIC;
        S, C: OUT STD_LOGIC
    );
END HALF_ADDER;

ARCHITECTURE HALF_ADDER_ARCH OF HALF_ADDER IS
BEGIN
    S <= A XOR B;
    C <= A AND B;
END HALF_ADDER_ARCH;

```

B. Behavior Style Architecture

The behavioral style architecture models how the circuit outputs will behave to the circuit inputs. This model may not reflect how the circuit is implemented when it is synthesized. Process statement is the core part of behavioral style architecture. In this style, the internal working is implemented using sequential statements within process statements.

Example:

```

LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

ENTITY HALF_ADDER IS
    PORT(
        A, B: IN STD_LOGIC;
        S, C: OUT STD_LOGIC
    );
END HALF_ADDER;

```

```

ARCHITECTURE HALF_ADDER_ARCH OF HALF_ADDER IS
BEGIN
    PROCESS_ADDER: PROCESS (A, B)
    BEGIN
        S <= A XOR B;
        C <= A AND B;
    END PROCESS PROCESS_ADDER;
END HALF_ADDER_ARCH;

```

C. Structural Style Architecture

The structural style architecture is a modular approach to coding which supports hierarchical design which is essential to understand complex digital designs. Modular designs enhance understandability by combining low-level functionality into modules. These modules can be reused in different designs resulting in save of design time. VHDL structural model may not be efficient for simple designs. However, the following are the general steps for writing structural model code.

- Initially the entity and architecture implementations for the individual gates or modules which are within our system must be defined.
- The entity declaration of our system is done, similar to other models.
- Different components used in our design are declared within the declarative part of architecture. Component declaration is similar to entity declaration, only keyword entity must be replaced by keyword component.
- Internal signals, which are the intermediate output signals of one module fed into another module as input signals, are declared.
- Finally, instances of all modules are created and mapped in the architecture body. Mapping can be done using direct mapping or implied mapping. In direct mapping, each of the internal signals and signals of entity of the system are directly associated with the signals of corresponding components. Whereas in implied mapping, only internal signals and signals of entity of the system are listed. Though it uses less space, but it requires the signals be placed in the proper order.

A FSM is specified by five entities: symbolic states, input signals, output signals, next-state function and output function. A state specifies a unique internal condition of a system and the FSM transits from one state to another with time. The next-state function is used to determine the next state of the system. The output function specifies the value of the output signals. The general block diagram of FSM is shown in the figure 10.1.

FSM consists of two sections; combinational and sequential logic. The combinational part has two inputs – external input and present state – and two outputs; next state and external output. Whereas, the sequential section has three inputs – clock, reset, and next state – and one output in a form of present state. Since the flip flops are implemented in sequential logic, clock and reset are part of this section.

If the output of the machine depends not only on the present state but also on the current input, then it is called a Mealy machine. Otherwise, if it depends only on the current state, it is called a Moore machine.

A. Design of Sequential Section

PROCESS statement is required for sequential section. The clock and reset signals appear in the sensitivity list of PROCESS statement. When reset is asserted, present state will be set to initial state of the system. In other cases, present state will change to next state at the proper clock edge. A typical design template for the sequential section is given as:

```
PROCESS (RESET, CLOCK)
BEGIN
    IF (RESET = '1') THEN
        PRESENT_STATE <= INITIAL_STATE;
    ELSIF (CLOCK'EVENT AND CLOCK = '1') THEN
        PRESENT_STATE <= NEXT_STATE;
    END IF;
END PROCESS;
```